

COMPLETE PYTHON NOTES

Topic • Definition • Syntax • Example (Question / Program / Output)

SYLLABUS / TABLE OF CONTENTS

1. Basics: Introduction to Python

2. Keywords

3. Identifiers

4. Comments

5. Variables

6. Operators

6.1 Arithmetic Operators

6.2 Comparison Operators

6.3 Logical Operators

6.4 Assignment Operators

7. Control Flow: Conditionals

7.1 'if' Statement

7.2 'elif' Statement

7.3 'else' Statement

8. Loops and Control Keywords

8.1 'while' Loop

8.2 'for' Loop

8.3 'break' Keyword

8.4 'continue' Keyword

9. Data Structures

9.1 Lists

9.2 Tuples

9.3 Dictionaries

9.4 Sets

9.5 Strings

10. Functions

10.1 User-Defined Functions

10.2 *args and **kwargs

10.3 Lambda Expressions

10.4 List Comprehensions

11. Object-Oriented Programming (OOP)

11.1 Classes and Objects

11.2 __init__ & self

11.3 Inheritance

11.4 Polymorphism

11.5 Encapsulation

11.6 Abstraction

12. Advanced Topics

12.1 File Operations

12.2 Exception Handling

12.3 Generators

12.4 Decorators

12.5 Regular Expressions (re)

12.6 Virtual Environments (venv) & pip

1. Basics: Introduction to Python

Definition:

- * Python is a high-level, interpreted programming language known for its clear syntax and great readability.
- * It is widely used in web development, data science, artificial intelligence, and automation tasks.

Syntax:

```
print("Your Message Here")
```

Example

Question: Write a program to print a welcome message.

Program:

```
print("Hello, Welcome to the world of Python programming!")
```

Output:

```
Hello, Welcome to the world of Python programming!
```

2. Keywords

Definition:

- * Keywords are reserved words in Python that have fixed meanings and cannot be used as regular identifiers.
- * They form the core vocabulary of the language to build control logic and definitions.

Syntax:

```
True, False, None, and, or, not, if, else, elif,  
for, while, break, continue, def, return, class, import
```

Example

Question: Use the keywords True, if and else to check a condition.

Program:

```
condition_flag = True  
if condition_flag:  
    print("The keyword condition is satisfied.")  
else:  
    print("Condition failed.")
```

Output:

```
The keyword condition is satisfied.
```

3. Identifiers

Definition:

- * Identifiers are user-defined names given to entities like variables, functions, classes, and modules to recognize them distinctly.
- * They must start with a letter or an underscore, followed by letters, digits, or underscores.

Syntax:

```
valid_name = value  
_valid_name2 = value
```

Example

Question: Create two valid identifiers and print their values.

Program:

```
student_age = 21  
_user_score = 94.5  
print(student_age, _user_score)
```

Output:

```
21 94.5
```

4. Comments

Definition:

- * Comments are non-executable explanatory notes written within the code to help developers read and understand the logic.
- * Python uses the hash symbol for single lines or triple quotes for multiline annotations.

Syntax:

```
# Single line comment  
''' Multi-line comment block '''
```

Example

Question: Show how single-line comments are used in code.

Program:

```
# This line prints a message  
print("Comments are hidden instructions") # inline comment
```

Output:

```
Comments are hidden instructions
```

5. Variables

Definition:

- * Variables are named storage locations or containers used to hold data values that can change during execution.
- * Python allocates memory dynamically, so you do not need to declare data types explicitly.

Syntax:

```
variable_name = data_value
```

Example

Question: Store a name and a rank in variables and print them.

Program:

```
user_name = "Praveen"  
user_rank = 10  
print(user_name, user_rank)
```

Output:

```
Praveen 10
```

6. Operators

6.1 Arithmetic Operators

Definition:

- * Arithmetic operators are standard mathematical symbols used to carry out numeric computations like addition, subtraction, multiplication, and division.

Syntax:

```
+, -, *, /, %, **, //
```

Example

Question: Add two numbers and find the power of a number.

Program:

```
sum_result = 15 + 5  
power_result = 2 ** 3  
print(sum_result, power_result)
```

Output:

```
20 8
```

6.2 Comparison Operators

Definition:

- * Comparison operators evaluate relationships between two operands and return a Boolean response of either True or False based on equality or size.

Syntax:

```
==, !=, >, <, >=, <=
```

Example

Question: Compare two pairs of numbers for equality and size.

Program:

```
check_equality = (10 == 10)
check_greater = (5 > 12)
print(check_equality, check_greater)
```

Output:

```
True False
```

6.3 Logical Operators

Definition:

- * Logical operators combine conditional assertions to evaluate compound Boolean conditions and direct logical outcomes.

Syntax:

and, or, not

Example

Question: Combine two conditions using logical operators.

Program:

```
combined_logic = (5 > 2) and (10 < 20)
inverted_logic = not(5 == 5)
print(combined_logic, inverted_logic)
```

Output:

True False

6.4 Assignment Operators

Definition:

- * Assignment operators give values to variables or update their existing storage values in combination with mathematical operations.

Syntax:

```
=, +=, -=, *=, /=
```

Example

Question: Assign a value and update it using +=.

Program:

```
score_tracker = 50  
score_tracker += 10  
print(score_tracker)
```

Output:

```
60
```

7. Control Flow: Conditionals

7.1 'if' Statement

Definition:

- * The 'if' statement runs a specific block of target code only when the provided evaluation expression tests as True.

Syntax:

```
if condition:  
    # code block to execute
```

Example

Question: Check if marks are 75 or above.

Program:

```
current_marks = 85  
if current_marks >= 75:  
    print("You passed with distinction marks!")
```

Output:

```
You passed with distinction marks!
```

7.2 'elif' Statement

Definition:

- * The 'elif' (else-if) statement tests alternative condition scenarios sequentially when the original 'if' logic fails.

Syntax:

```
elif alternate_condition:  
    # code block to execute
```

Example

Question: Grade a student using if and elif.

Program:

```
current_marks = 65  
if current_marks >= 75:  
    print("Distinction grade")  
elif current_marks >= 60:  
    print("First class grade")
```

Output:

```
First class grade
```

7.3 'else' Statement

Definition:

- * The 'else' block serves as a default catch-all that runs when none of the preceding conditional statements are True.

Syntax:

```
else:  
# code block to execute
```

Example

Question: Use else as a fallback when marks are below 50.

Program:

```
current_marks = 45  
if current_marks >= 50:  
    print("Passed successfully")  
else:  
    print("Failed the examination")
```

Output:

```
Failed the examination
```

8. Loops and Control Keywords

8.1 'while' Loop

Definition:

- * A 'while' loop runs a target block of instructions repeatedly as long as its guiding conditional check remains True.

Syntax:

```
while condition:  
    # code block to repeat
```

Example

Question: Print numbers 1 to 3 using a while loop.

Program:

```
step_counter = 1  
while step_counter <= 3:  
    print("Loop iteration step:", step_counter)  
    step_counter += 1
```

Output:

```
Loop iteration step: 1  
Loop iteration step: 2  
Loop iteration step: 3
```

8.2 'for' Loop

Definition:

- * The 'for' loop iterates over sequential structures like lists, tuples, or ranges, running the block for each element.

Syntax:

```
for item in sequence:  
    # code block to repeat
```

Example

Question: Print numbers 1 to 3 using a for loop and range().

Program:

```
for current_number in range(1, 4):  
    print("Sequence value index is:", current_number)
```

Output:

```
Sequence value index is: 1  
Sequence value index is: 2  
Sequence value index is: 3
```

8.3 'break' Keyword

Definition:

- * The 'break' statement terminates the active execution loop early and shifts the program flow to the next line outside the loop.

Syntax:

```
break
```

Example

Question: Stop the loop as soon as the value reaches 4.

Program:

```
for item_val in range(1, 10):  
    if item_val == 4:  
        break  
    print("Item index counter:", item_val)
```

Output:

```
Item index counter: 1  
Item index counter: 2  
Item index counter: 3
```

8.4 'continue' Keyword

Definition:

- * The 'continue' statement skips the remaining code in the current iteration and jumps directly to the loop's next evaluation.

Syntax:

```
continue
```

Example

Question: Skip printing the number 3 but continue the loop.

Program:

```
for numeric_val in range(1, 5):  
    if numeric_val == 3:  
        continue  
    print("Active structural digit:", numeric_val)
```

Output:

```
Active structural digit: 1  
Active structural digit: 2  
Active structural digit: 4
```

9. Data Structures

9.1 Lists

Definition:

- * Lists are mutable, ordered collections of items enclosed in square brackets that allow duplicate records and direct index changes.

Syntax:

```
list_name = [item1, item2, item3]
```

Example

Question: Create a list of fruits and add a new item.

Program:

```
fruits_list = ["apple", "banana"]  
fruits_list.append("cherry")  
print("Updated target list object:", fruits_list)
```

Output:

```
Updated target list object: ['apple', 'banana', 'cherry']
```

9.2 Tuples

Definition:

- * Tuples are immutable, ordered sequences wrapped inside standard parentheses that protect data from accidental updates after creation.

Syntax:

```
tuple_name = (item1, item2, item3)
```

Example

Question: Store a coordinate pair as a tuple and access an item.

Program:

```
coordinate_point = (12.5, 45.8)
print("Fixed system coordinate details:",
      coordinate_point[0])
```

Output:

```
Fixed system coordinate details: 12.5
```

9.3 Dictionaries

Definition:

- * Dictionaries are unordered, mutable structures that store elements as key-value pairs inside curly braces for fast lookups.

Syntax:

```
dict_name = {"key": "value"}
```

Example

Question: Store a user profile as key-value pairs and fetch a value.

Program:

```
user_profile = {"id": 101, "role": "Admin"}  
print("Profile assigned role designation:",  
      user_profile["role"])
```

Output:

```
Profile assigned role designation: Admin
```

9.4 Sets

Definition:

- * Sets are unordered collections of unique elements wrapped in curly braces that filter out duplicates and handle mathematical operations.

Syntax:

```
set_name = {item1, item2}
```

Example

Question: Remove duplicate numbers automatically using a set.

Program:

```
unique_digits = {1, 2, 2, 3}  
print("Clean unique set collection values:", unique_digits)
```

Output:

```
Clean unique set collection values: {1, 2, 3}
```

9.5 Strings

Definition:

- * Strings are immutable sequences of text characters wrapped inside quotation marks, equipped with built-in manipulation methods.

Syntax:

```
string_name = "Text Characters Here"
```

Example

Question: Convert a greeting string to uppercase.

Program:

```
greeting_text = "hello"  
print("Capitalized string modification:",  
      greeting_text.upper())
```

Output:

```
Capitalized string modification: HELLO
```

10. Functions

10.1 User-Defined Functions

Definition:

- * User-defined functions are modular, reusable blocks of code declared with the 'def' keyword to perform a specific task when called.

Syntax:

```
def function_name(parameters):  
    return result
```

Example

Question: Write a function that returns the square of a number.

Program:

```
def calculate_square(input_val):  
    return input_val * input_val  
  
print("Function call computation output:",  
      calculate_square(5))
```

Output:

```
Function call computation output: 25
```

10.2 *args and **kwargs

Definition:

- * `*args` accepts a variable number of positional arguments as a tuple.
- * `**kwargs` accepts a variable number of keyword arguments as a dictionary.

Syntax:

```
def function_name(*args, **kwargs):
```

Example

Question: Write a function that accepts flexible positional and keyword arguments.

Program:

```
def display_flexible_inputs(*args, **kwargs):  
    print("Positional arguments tuple:", args)  
    print("Keyword arguments dictionary:", kwargs)  
  
    display_flexible_inputs(1, 2, status="Active")
```

Output:

```
Positional arguments tuple: (1, 2)  
Keyword arguments dictionary: {'status': 'Active'}
```

10.3 Lambda Expressions

Definition:

- * Lambda expressions are small, anonymous, single-line functions that can take multiple arguments but evaluate only a single expression.

Syntax:

```
lambda arguments: expression
```

Example

Question: Write a one-line lambda function to multiply two numbers.

Program:

```
multiply_nums = lambda a, b: a * b
print("Lambda expression multiplication:", multiply_nums(4,
5))
```

Output:

```
Lambda expression multiplication: 20
```

10.4 List Comprehensions

Definition:

- * List comprehensions provide a concise syntax to create new lists from existing collections or iterables based on an inline loop.

Syntax:

```
[expression for item in iterable]
```

Example

Question: Build a list of squares for numbers 1 to 3 in one line.

Program:

```
squared_evens = [x*x for x in range(1, 4)]  
print("List comprehension result:", squared_evens)
```

Output:

```
List comprehension result: [1, 4, 9]
```

11. Object-Oriented Programming (OOP)

11.1 Classes and Objects

Definition:

- * A class is a structural blueprint or template for creating objects, which are individual, functional instances of that class.

Syntax:

```
class ClassName:  
    pass  
object_name = ClassName()
```

Example

Question: Create a simple class and instantiate an object from it.

Program:

```
class CarBlueprint:  
    pass  
  
my_car_object = CarBlueprint()  
print("Object created:", my_car_object)
```

Output:

```
Object created: <__main__.CarBlueprint object at 0x...>
```

11.2 __init__ & self

Definition:

- * The `__init__` method is Python's constructor that initializes an object's attributes.
- * 'self' represents the specific instance being created.

Syntax:

```
def __init__(self, parameters):  
    self.attribute = parameters
```

Example

Question: Initialize an object's attribute using the constructor.

Program:

```
class DeviceProfile:  
    def __init__(self, core_name):  
        self.device_name = core_name  
  
active_phone = DeviceProfile("SmartPhone")  
print("Device name:", active_phone.device_name)
```

Output:

```
Device name: SmartPhone
```

11.3 Inheritance

Definition:

- * Inheritance allows a child class to inherit attributes and methods from a parent class, promoting code reuse and extensions.

Syntax:

```
class ChildClass(ParentClass):  
    pass
```

Example

Question: Let a child class use a method from its parent class.

Program:

```
class AnimalParent:  
    def breathe_action(self):  
        print("Breathing active")  
  
class DogChild(AnimalParent):  
    pass  
  
my_dog = DogChild()  
my_dog.breathe_action()
```

Output:

```
Breathing active
```

11.4 Polymorphism

Definition:

- * Polymorphism allows different classes to share the same method name but implement their own specific behaviors.

Syntax:

```
# Multiple classes implementing the same method name
```

Example

Question: Give two classes the same method name with different behavior.

Program:

```
class Truck:
    def drive(self):
        return "Driving a heavy vehicle"

class Bike:
    def drive(self):
        return "Riding a light bicycle"

print(Truck().drive())
print(Bike().drive())
```

Output:

```
Driving a heavy vehicle
Riding a light bicycle
```

11.5 Encapsulation

Definition:

- * Encapsulation restricts direct access to an object's components by prefixing attribute names with underscores to prevent external modifications.

Syntax:

```
self.__private_variable = value
```

Example

Question: Hide an attribute using a double underscore prefix.

Program:

```
class SecureVault:
    def __init__(self):
        self.__secret_key = "99XF"
    def reveal(self):
        return self.__secret_key

my_vault = SecureVault()
print("Secret via method:", my_vault.reveal())
```

Output:

```
Secret via method: 99XF
```

11.6 Abstraction

Definition:

- * Abstraction hides complex internal implementation details and only exposes essential features using abstract base classes and methods.

Syntax:

```
from abc import ABC, abstractmethod
```

Example

Question: Force a subclass to implement an abstract method.

Program:

```
from abc import ABC, abstractmethod

class AbstractForm(ABC):
    @abstractmethod
    def click_action(self):
        pass

class SubmitButton(AbstractForm):
    def click_action(self):
        return "Form submitted"

print(SubmitButton().click_action())
```

Output:

```
Form submitted
```

12. Advanced Topics

12.1 File Operations

Definition:

- * File operations allow you to open, read, write, and close physical external files stored on the computer disk safely.

Syntax:

```
with open("filename.txt", "mode") as file:  
    # code
```

Example

Question: Write a line of text into a file safely.

Program:

```
with open("test_file.txt", "w") as target_file:  
    target_file.write("Python writing sample.")  
    print("File written successfully")
```

Output:

```
File written successfully
```

12.2 Exception Handling

Definition:

- * Exception handling isolates code blocks using try, except, and finally to catch runtime errors gracefully without crashing the app.

Syntax:

```
try:  
    # code  
except Exception as e:  
    # handle error
```

Example

Question: Catch a division-by-zero error gracefully.

Program:

```
try:  
    division_calc = 10 / 0  
except ZeroDivisionError:  
    print("Cannot compute division by zero!")
```

Output:

```
Cannot compute division by zero!
```

12.3 Generators

Definition:

- * Generators are memory-efficient functions that return an iterator loop stream sequentially using the 'yield' keyword instead of 'return'.

Syntax:

```
def generator_name():  
    yield value
```

Example

Question: Create a generator that yields numbers on demand.

Program:

```
def stream_numbers():  
    yield 1  
    yield 2  
  
number_iterator = stream_numbers()  
print(next(number_iterator))
```

Output:

```
1
```

12.4 Decorators

Definition:

- * Decorators are wrappers that dynamically modify or extend the behavior of a target function without modifying its permanent source code.

Syntax:

```
@decorator_name  
def target_function():  
    pass
```

Example

Question: Wrap a function to add a log message before it runs.

Program:

```
def log_decorator(original_func):  
    def wrapper_logic():  
        print("Log message text before call")  
        original_func()  
    return wrapper_logic  
  
@log_decorator  
def say_hi():  
    print("Hi!")  
  
say_hi()
```

Output:

```
Log message text before call  
Hi!
```

12.5 Regular Expressions (re)

Definition:

- * Regular Expressions (re) provide search parameters and matching tools to scan, filter, and modify text strings based on specific character patterns.

Syntax:

```
import re
match = re.search(pattern, string)
```

Example

Question: Extract all digit sequences from a text string.

Program:

```
import re
text_block = "User phone code is 4567"
found_digits = re.findall(r"\d+", text_block)
print("Extracted pattern digits list:", found_digits)
```

Output:

```
Extracted pattern digits list: ['4567']
```

12.6 Virtual Environments (venv) & pip

Definition:

- * Virtual environments isolate dependencies for individual projects.
- * Pip downloads and manages external third-party libraries.

Syntax:

```
python -m venv environment_name  
pip install package_name
```

Example

Question: Show the terminal commands to create a venv and install a package.

Program:

```
# python -m venv development_env  
# pip install requests  
print("Environment setup commands shown above")
```

Output:

```
Environment setup commands shown above
```